Statement of Work (SOW)
Deployment of LLM

Date:

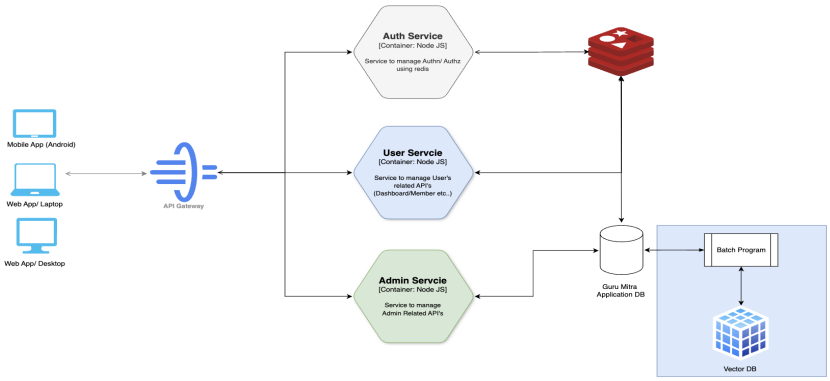
27th June 2025

1. About LLM

The **LLM** project aims to build an AI-powered platform that empowers teachers to enhance their teaching practices, boost student engagement, and support ongoing professional development. The platform is being expanded to include advanced features such as comprehensive student performance tracking for classes 6 to 8 across all subjects, support for regional languages like Telugu, and AI-driven insights for personalized content recommendations. With the integration of MILA resources, a self-improving knowledge base, and mechanisms for capturing high-priority school feedback, LLM is designed to continuously evolve—ensuring it remains responsive to the real needs of educators while fostering a more effective and user-centric learning environment.

2. Architecture

LLM is a teacher assistant platform designed to support educators by leveraging AI-powered Large Language Models (LLMs) for generating lesson plans, quizzes, and assignments. The platform follows a microservices-based architecture with a Node.js backend and a cross-platform frontend built using React.js and the Ionic framework. The system is designed for scalability, high performance, and ease of access across multiple devices.



Frontend Architecture

Technology Stack:

- **React.js**: Used for building web applications.
- **Ionic Framework**: Enables cross-platform support, allowing a single codebase to work seamlessly on Web, Progressive Web App (PWA), and Android.

Supported Platforms:

- Web Application (Desktop and Laptop)
- Progressive Web App (PWA)
- Android Mobile Application

The frontend communicates with the backend through an **API Gateway**, ensuring secure and efficient routing of requests.

Backend Architecture

The backend follows a **Node.js-based microservices architecture** with distinct services handling different functionalities.

Key Components:

1. **API Gateway**:
    - Serves as the entry point for all client requests.
    - Ensures secure communication and request routing.
    - Manages authentication and authorization by interacting with the Auth Service.
2. **Auth Service (Node.js, Redis)**:
    - Handles authentication and authorization.
    - Uses Redis for session management and caching user authentication tokens to enhance performance.
3. **User Service (Node.js)**:
    - Manages user-related APIs, including profile management, dashboards, and membership-related functionalities.
    - Interacts with the LLM Application Database to fetch and update user-related data.
4. **Admin Service (Node.js)**:
    - Manages admin-specific APIs.
    - Allows administrative users to control system configurations, manage users, and monitor system activity.

Data Flow and Storage

LLM uses multiple data storage solutions to optimize performance and scalability:

1. Vector Database:

- Used to store AI-generated lesson plans, quizzes, and assignments.
- Facilitates quick retrieval of relevant educational content using LLMs.

2. LLM Application Database (MongoDB):

- Stores structured application data, including user information, session history, and administrative records.
- Synchronizes with Vector DB to replicate AI-generated content.

3. Redis Cache:

- Used to store frequently accessed data for faster retrieval.
- Reduces database load by caching user authentication tokens, frequently used lesson plans, and quiz templates.

Batch Processing Module:

- Periodically extracts data from the **Vector DB** and replicates it into **MongoDB** and **Redis**.
- Ensures optimal system performance by keeping frequently accessed AI-generated content readily available in cache and primary databases.

System Flow

1. Users interact with the **frontend application** (React.js + Ionic Framework) via web, PWA, or Android apps.
2. Requests are routed through the **API Gateway**, which forwards them to the appropriate microservices.
3. The **Auth Service** authenticates and authorizes requests using Redis.
4. Authenticated requests are handled by either the **User Service** or **Admin Service**, depending on the request type.
5. The backend services interact with the **LLM Application Database (MongoDB)** to store and retrieve user-related data.
6. AI-generated content is fetched from the **Vector Database**, and a **Batch Program** periodically updates MongoDB and Redis with the latest AI-generated lesson plans and quizzes.
7. Users receive optimized responses via the frontend application.

3. Scope of Work

The vendor will be responsible for:

The **LLM** platform is being developed with a cloud-agnostic architecture to ensure scalability, flexibility, and ease of deployment across different cloud environments. The application is first deployed in a **User Acceptance Testing (UAT)** environment, allowing educators and stakeholders to validate features, provide feedback, and ensure alignment with academic needs and operational expectations. Following successful testing and refinements, the platform is deployed in a **Production-ready environment** with high availability, autoscaling capabilities, and secure integration with AI services.

This multi-stage deployment strategy ensures minimal downtime, robust testing, and seamless transition from UAT to PROD. By maintaining compatibility with major cloud providers (such as Azure, AWS, GCP or any cloud service provider), LLM remains adaptable to diverse institutional IT policies and infrastructures, supporting its vision of delivering an intelligent, teacher-first platform at scale.

A. Infrastructure Setup

- Provision and configure Azure Kubernetes Service (AKS) cluster (6-node, autoscaling enabled)
- Setup Cosmos DB (Mongo API), Azure Redis Cache
- Configure Azure Front Door, Load Balancer, and App Gateway
- Setup Virtual Networks, Subnets, and Private Endpoints

B. Security and Compliance

- Implement Azure Key Vault for secrets management
- Configure Network Security Groups (NSGs), Azure Firewall, and Azure DDoS Protection
- Enable Azure Defender for Cloud and role-based access control (RBAC)

C. Application Deployment

- Containerize application (frontend and backend)
- Setup CI/CD pipelines using GitHub Actions or Azure DevOps
- Deploy AI Assistant and TeachWise services on AKS

- Integrate GPT-4.0 Mini API for AI services

D. Monitoring and Logging

- Enable Azure Monitor and Log Analytics
- Setup alerts, dashboards, and performance logs

E. Documentation & Handover

- Provide deployment documentation, access matrix, security configurations, and SOPs
- Handover to internal IT and support teams

4. Deliverables

- Fully deployed and functional LLM production environment on Azure
- CI/CD pipelines integrated with version control
- Secured AKS-based deployment with autoscaling
- Monitoring and alerting setup
- Final documentation and handover report

5. Additional Details

Service Name	Exposed Port(s)	Internal Port	Purpose
Auth Backend	5001/tcp	5001	Handles authentication and tokens
Admin Frontend	80/tcp	80	Admin dashboard UI
App Frontend	80/tcp	80	Teacher/user-facing frontend
Admin Backend	5003/tcp	5003	Admin backend services
User Backend	5002/tcp	5002	User backend APIs
HAProxy (Reverse Proxy)	80, 443 (host mapped)	80, 443	Routes traffic to frontend/backend
MinIO (Object Store)	9000–9001	9000–9001	File/object storage
MongoDB	27017	27017	Primary database

Redis	6379	6379	Caching and session management
Qdrant (Vector DB)	6333, 6334	6333, 6334	Vector database for embeddings